

Name:- Hamza Hashmi

Roll no:- A-35

Branch:- AI&DS (A)

Batch:- A2

Aim:- Use Sequence data types and their associated operations in python

a.Strings b.Lists c.Arrays d.Tuples e.Dictionary f.Sets g.Range

1. Sequence Data Types in Python Sequence data types are collections used to store data in a structured, ordered format. Python provides several built-in sequences:

Strings (str): Immutable sequences of Unicode characters.

Lists (list): Mutable, ordered collections of items.

Tuples (tuple): Immutable, ordered collections of items.

Ranges (range): Immutable sequences of numbers, usually used in loops.

Indexing and Slicing Indexing and slicing are universal across these types because they rely on the order of elements.

Indexing: Accessing a single element using its position. Python uses 0-based indexing. You can also use negative indexing (e.g., -1 for the last element).

Slicing: Accessing a range of elements using the syntax [start:stop:step]. The stop index is exclusive.

2. Lists vs. Tuples

The fundamental difference lies in mutability.

List

- Mutable (Can be changed)

- Square brackets []

- Slightly slower (extra memory overhead)

Tuple

- Immutable (Cannot be changed)

- Parentheses ()

- Faster (fixed memory size)

Why Prefer a Tuple?

Data Integrity: Use tuples for data that should never change (like coordinates or configuration constants).

Dictionary Keys: Since tuples are immutable and "hashable," they can be used as keys in a dictionary; lists cannot.

Speed: Iterating through a tuple is faster than a list due to Python's internal memory optimization for fixed-size objects.

3. Python Arrays vs. Lists

In standard Python, "lists" are often used where other languages use "arrays," but there is a dedicated array module.

Type Flexibility: Lists can store multiple data types (integers, strings, and floats together). Arrays require all elements to be of the same data type (e.g., all integers).

Memory Efficiency: Arrays are more memory-efficient because they store data as raw machine values rather than Python objects

4. How Dictionaries Work Internally

Python dictionaries are implemented using Hash Tables.

Hashing: When you add a key-value pair, Python passes the key through a hash function to generate a unique integer (a "hash").

Bucket Mapping: This hash is used to determine a specific "bucket" or index in an internal array where the value will be stored.

Why are lookups faster?

In a list, to find an item, Python may have to check every single element (an $O(n)$ operation). In a dictionary, Python hashes the key and goes directly to the memory address where the value is stored. This makes the lookup time constant ($O(1)$), regardless of how large the

dictionary grows.

5. Sets vs. Lists

The main difference is that sets are unordered and contain only unique elements.

List

-Preserves insertion order.

-Allows duplicates.

-Accessed via index.

Set

-Unordered.

-No duplicates allowed.

-No indexing/slicing.

```
text = "PythonProgramming"
print("Length of string:", len(text))
print("First three characters:", text[:3])
```

```
Length of string: 17
First three characters: Pyt
```

```
numbers = [45, 12, 89, 7, 33]
print("List:", numbers)
print("Maximum value:", max(numbers))
print("Minimum value:", min(numbers))
```

```
List: [45, 12, 89, 7, 33]
Maximum value: 89
Minimum value: 7
```

```
students = ("Alice", "Bob", "Charlie", "David")
print("Student Names:")
for name in students:
    print("-", name)
```

```
Student Names:
- Alice
- Bob
- Charlie
- David
```

```
# Roll Number (Key) : Name (Value)
student_records = {101: "Aryan", 102: "Snehal", 103: "John"}
print("Dictionary Keys (Roll Numbers):", student_records.keys())
```

```
Dictionary Keys (Roll Numbers): dict_keys([101, 102, 103])
```

```
raw_list = [1, 2, 2, 3, 4, 4, 4, 5, 6, 6]
# Converting to a set automatically removes duplicates
unique_set = set(raw_list)
print("Original List:", raw_list)
print("Unique Set:", unique_set)
```

```
Original List: [1, 2, 2, 3, 4, 4, 4, 5, 6, 6]
Unique Set: {1, 2, 3, 4, 5, 6}
```

```
input_str = "Hello World"
vowels = "aeiouAEIOU"
count = 0

for char in input_str:
    if char in vowels:
        count += 1

print(f"Number of vowels in '{input_str}':", count)
```

```
Number of vowels in 'Hello World': 3
```

```
nums = [10, 20, 30, 40]

# Insertion
nums.insert(2, 25) # Inserts 25 at index 2
nums.append(50)    # Adds 50 to the end
print("After insertion:", nums)

# Deletion
nums.pop(1)        # Removes element at index 1 (20)
nums.remove(40)    # Removes the specific value 40
print("After deletion:", nums)
```

```
After insertion: [10, 20, 25, 30, 40, 50]
After deletion: [10, 25, 30, 50]
```

```
my_list = [1, 2, 3, 4]
my_tuple = ("A", "B", "C")

# List to Tuple
converted_tuple = tuple(my_list)
print("Converted to Tuple:", converted_tuple)

# Tuple to List
converted_list = list(my_tuple)
print("Converted to List:", converted_list)
```

```
Converted to Tuple: (1, 2, 3, 4)
Converted to List: ['A', 'B', 'C']
```

Conclusion

This practical demonstrates the versatility of Python's built-in data structures. By implementing these exercises, we can conclude that:

- Sequences (Strings, Lists, Tuples, Range): These are essential for storing ordered data. While Lists offer flexibility through mutability (allowing insertion and deletion), Tuples provide data integrity and performance benefits due to their immutability.
- Collections (Sets, Dictionaries): These are optimized for specific tasks beyond simple ordering. Sets are the most efficient way to handle unique elements and perform mathematical operations, while Dictionaries provide high-speed data retrieval using key-value pairs.
- Versatility: Python's ability to easily convert between these types (e.g., changing a List to a Set to remove duplicates) makes it a powerful language for data manipulation and text processing.